

Chapter 10

java.util Package & Collection Framework

1. What Is the java.util Package?

The java.util package is one of the most important built-in packages in Java. It provides utility classes that help in:

- Data storage (Collections)
- Working with dates
- Working with random numbers
- Reading input
- Handling arrays
- Using scanners, lists, maps, sets, etc.

It contains tools that make Java programming easier and more powerful.

Why Do We Need java.util?

Because core Java does not have built-in features for many tasks like:

- Storing multiple objects easily (ArrayList, HashMap)
- Sorting data (Collections.sort)
- Getting user input (Scanner)
- Working with dates and times (Date, Calendar)
- Generating random numbers (Random)

So Java provides these utility classes inside the java.util package.

Common Classes in java.util Package

Class Name	Purpose
Scanner	To take input from user
ArrayList	Growable array
HashMap	Key-value pair map
HashSet	Set of unique values
LinkedList	List with nodes
Collections	Utility methods (sort, reverse...)
Date	Work with dates
Random	Generate random values
Arrays	Utility methods for arrays

2. What Is the Collections Framework?

The Java Collections Framework is a set of classes and interfaces that help store, manage, and process groups of objects efficiently.

- Resizable data structures
- Built-in algorithms (sorting, searching, etc.)

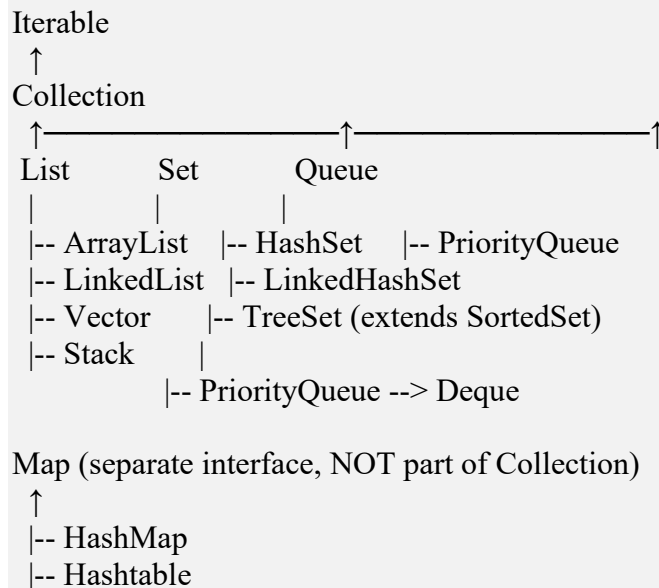
- Easy data processing
- Provides ready-made classes like ArrayList, HashSet, HashMap

Why Collections? (Solving the Limitations of Arrays)

Arrays	Collections
Fixed size	Grow/shrink dynamically
Can store only same type	Can store any reference type
No built-in algorithms	Has sorting, searching, etc.
Manual operations	Ready-made utility methods

3. The Collection Framework Hierarchy

java.util.*



Interfaces that extend **Collection**:

- **List** (ordered, duplicates allowed)
- **Set** (no duplicates)
- **Queue** (FIFO behavior)

Important note: **Map** is NOT part of the **Collection** interface. **Map** is a separate interface in the Java Collection Framework.

4. Collections (the Utility Class)

Collections (with an 's') is a utility class in the Java Collection Framework, located in `java.util.Collections`. It contains static helper methods that make it easier to work with collection objects like **List**, **Set**, and **Queue**.

- **Collections** = helper toolbox — it contains ready-made methods to operate on collections.
- **Collection** (interface) -> a group of objects.
- **Collections** (class) -> tools to work with those groups.

Difference Between Collection and Collections

Feature	Collection	Collections
Type	Interface	Utility (helper) class
Package	java.util.Collection	java.util.Collections
Purpose	To represent a group of objects	To provide utility methods for working with collections
Contains	Abstract methods (add, remove, size, etc.)	Static methods (sort, reverse, shuffle, etc.)
Used for	Creating collection objects (List, Set, Queue)	Performing operations on collection objects
Inheritance	Extended by List, Set, Queue	Cannot be extended; final class
Object creation	You cannot create a Collection directly (interface)	You cannot create an object; methods are static
Examples	Collection<String> c = new ArrayList<>();	Collections.sort(list);
Part of hierarchy?	Yes, core part of Collection Framework hierarchy	No, it sits outside the hierarchy as a helper class
Supports Generics?	Yes	Yes (through methods)

Key Uses of the Collections Class

```
import java.util.Collections;
```

Task	Example
Sorting	Collections.sort(list);
Reversing a list	Collections.reverse(list);
Shuffling	Collections.shuffle(list);
Finding maximum / minimum	Collections.max(list); / Collections.min(list);
Making a collection thread-safe	Collections.synchronizedList(list);
Making a collection read-only	Collections.unmodifiableList(list);

Where Is Collections Used?

- Storing records (students, employees)
- Handling data from databases
- Managing lists in real apps (shopping carts)
- Processing data in algorithms

5. The Collection Interface — Syntax

```
// Importing Collection
import java.util.Collection;

// Creating a Collection (using a class like ArrayList)
Collection<String> names = new ArrayList<>();

// Common Methods
names.add("John");
```

```
names.remove("John");
names.size();
names.clear();
names.contains("John");
```

6. Difference Between List, Set, and Map

Feature	List	Set	Map
Stores	Values	Unique values	Key-value pairs
Order	Ordered	Unordered (except LinkedHashSet/TreeSet)	Keys unordered (depends on type)
Duplicates	Allowed	Not allowed	Keys must be unique
Access	Via index	No index	Via key
Examples	ArrayList, LinkedList	HashSet, TreeSet	HashMap, TreeMap

7. List Interface

List is a subinterface of Collection in Java. It represents an ordered collection where:

- Duplicates are allowed
- Elements can be accessed by index (0-based)
- Maintains insertion order

Syntax: `List<DataType> listName = new ArrayList<>();`

Declaring Lists of Different Types

```
List<String> names = new ArrayList<>();
List<Integer> numbers = new ArrayList<>();
List<Student> students = new ArrayList<>();
```

Adding and Accessing Elements

```
List<String> names = new ArrayList<>();
names.add("John");
names.add("Alice");
```

Simple Example — Full Program

```
import java.util.*;

public class ListExample {
    public static void main(String[] args) {
        List<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");
        fruits.add("Banana"); // duplicate allowed

        System.out.println(fruits);
    }
}
```

```
}  
}
```

Output: [Apple, Banana, Orange, Banana]

Accessing Elements Using Index

```
List<Integer> numbers = new ArrayList<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);
```

```
System.out.println(numbers.get(1)); // Output: 20
```

```
// Enhanced for loop  
for (String fruit : fruits) {  
    System.out.println(fruit);  
}
```

```
// Regular for loop with index  
for (int i = 0; i < fruits.size(); i++) {  
    System.out.println(fruits.get(i));  
}
```

8. List Implementations

8.1 ArrayList

What is it?

- A dynamic array that grows automatically.
- Allows duplicates and maintains insertion order.
- Fast when accessing elements (index-based access).
- Slower when inserting/deleting in the middle.

ArrayList Method Reference

Method	Description
add(E e)	Add element
add(int index, E e)	Add at index
get(int index)	Get element
set(int index, E e)	Replace element
remove(int index)	Remove at index
remove(Object o)	Remove by value
size()	Number of elements
contains(Object o)	Check if exists
isEmpty()	Check empty
clear()	Remove all elements

ArrayList Example

```

import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");
        list.add("Banana");
        list.add("Mango");
        list.add(1, "Grapes");

        System.out.println("List: " + list);
        System.out.println("Element at index 2: " + list.get(2));

        list.remove("Banana");
        System.out.println("After removing Banana: " + list);

        list.set(1, "Orange");
        System.out.println("After set: " + list);

        System.out.println("Size: " + list.size());
    }
}

```

List: [Apple, Grapes, Banana, Mango]

Element at index 2: Banana

After removing Banana: [Apple, Grapes, Mango]

After set: [Apple, Orange, Mango]

Size: 3

8.2 LinkedList

What is it?

- Based on a doubly-linked list.
- Best for insertion and deletion.
- Slower for random access (get()).

A doubly linked list is a linked list where each node contains data and two pointers — one pointing to the next node and another pointing to the previous node.

LinkedList Has All List Methods, Plus:

Method	Description
addFirst(E e)	Add at beginning
addLast(E e)	Add at end
getFirst()	First element
getLast()	Last element
removeFirst()	Remove first
removeLast()	Remove last

offer(E e)	Queue insert
poll()	Queue remove

LinkedList Example

```
import java.util.LinkedList;
```

```
public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();

        list.add("A");
        list.add("B");
        list.add("C");
        list.addFirst("Start");
        list.addLast("End");

        System.out.println("List: " + list);

        list.removeFirst();
        list.removeLast();

        System.out.println("After removing first & last: " + list);

        System.out.println("First element: " + list.getFirst());
        System.out.println("Last element: " + list.getLast());
    }
}
```

List: [Start, A, B, C, End]

After removing first & last: [A, B, C]

First element: A

Last element: C

8.3 Vector

Vector is a dynamic array similar to ArrayList but synchronized, meaning it is thread-safe.

- Thread-safe (synchronized)
- Slower than ArrayList due to synchronization
- Maintains insertion order
- Allows duplicates
- Grows by doubling its size when needed
- Legacy class (older, used before ArrayList)

Vector Method Reference

Method	Description
capacity()	Current capacity

addElement(E obj)	Add element
elementAt(int index)	Get element
removeElement(Object o)	Remove by value
removeAllElements()	Clear all

Vector Example

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
        Vector<Integer> v = new Vector<>();

        v.add(10);
        v.add(20);
        v.add(30);

        System.out.println("Vector: " + v);
        System.out.println("Element at 1: " + v.elementAt(1));

        v.removeElement(20);
        System.out.println("After remove: " + v);

        System.out.println("Capacity: " + v.capacity());
    }
}
```

Vector: [10, 20, 30]

Element at 1: 20

After remove: [10, 30]

Capacity: 10 (Vector's default initial capacity)

8.4 Stack

Stack is a subclass of Vector that follows the LIFO (Last In, First Out) order for storing and retrieving elements.

- Uses LIFO (Last In, First Out)
- Supports push, pop, peek operations
- Useful for undo, recursion, expression evaluation
- Thread-safe because it inherits from Vector
- Slower due to synchronization
- Legacy class (modern alternative: Deque)

Stack Method Reference

Method	Description
push(E e)	Add element
pop()	Remove & return top element

peek()	Return top without removing
empty()	Check if empty
search(Object o)	Position from top

Stack Example

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<String> stack = new Stack<>();

        stack.push("A");
        stack.push("B");
        stack.push("C");

        System.out.println("Stack: " + stack);
        System.out.println("Top element: " + stack.peek());

        stack.pop();
        System.out.println("After pop: " + stack);

        System.out.println("Is empty? " + stack.empty());
    }
}
```

Stack: [A, B, C]

Top element: C

After pop: [A, B]

Is empty? false

9. Set Interface

Set is a subinterface of Collection in Java that represents a group of unique elements.

Key Properties

- Does NOT allow duplicates
- Does NOT maintain insertion order (except LinkedHashSet)
- At most one null element (TreeSet does NOT allow null)

Syntax: Set<Type> set = new HashSet<>();

Common Implementations of Set

Implementation	Features
----------------	----------

HashSet	Fastest; no order maintained
LinkedHashSet	Maintains insertion order
TreeSet	Sorted order; no null values

Simple Set Example — HashSet

```
import java.util.*;

public class SetExample {
    public static void main(String[] args) {
        Set<String> names = new HashSet<>();

        names.add("John");
        names.add("Alice");
        names.add("Bob");
        names.add("Alice"); // duplicate ignored

        System.out.println(names);
    }
}
```

Output (order not guaranteed): [Bob, John, Alice]

Using LinkedHashSet (Maintains Order)

```
Set<Integer> numbers = new LinkedHashSet<>();

numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(20); // duplicate ignored

System.out.println(numbers); // [10, 20, 30]
```

Using TreeSet (Sorted Set)

```
Set<Integer> values = new TreeSet<>();

values.add(50);
values.add(10);
values.add(40);

System.out.println(values); // [10, 40, 50]
```

Important Methods — Common to All Set Types

Method	Description
add(E e)	Adds an element (ignored if duplicate)
remove(Object o)	Removes element
contains(Object o)	Checks if element exists
size()	Returns number of elements

<code>clear()</code>	Removes all elements
<code>isEmpty()</code>	Checks if empty
<code>iterator()</code>	Iterates through the set

10. Set Implementations

10.1 HashSet (Unordered, Faster)

- Stores elements in random order
- Fastest set implementation
- Allows null value

```
import java.util.*;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> hs = new HashSet<>();

        hs.add("Apple");
        hs.add("Banana");
        hs.add("Orange");
        hs.add("Apple"); // duplicate, ignored

        System.out.println("HashSet: " + hs);
        System.out.println("Contains Banana? " + hs.contains("Banana"));
        System.out.println("Size: " + hs.size());
    }
}
```

HashSet: [Orange, Banana, Apple] // order may vary

Contains Banana? true

Size: 3

10.2 LinkedHashSet (Maintains Insertion Order)

- Keeps elements in the same order they were inserted.
- Slightly slower than HashSet.

```
import java.util.*;

public class LinkedHashSetExample {
    public static void main(String[] args) {
        LinkedHashSet<Integer> lhs = new LinkedHashSet<>();

        lhs.add(40);
        lhs.add(10);
        lhs.add(30);
        lhs.add(10); // duplicate
    }
}
```

```

        System.out.println("LinkedHashSet: " + lhs);
    }
}

```

Expected Output: LinkedHashSet: [40, 10, 30]

10.3 TreeSet (Sorted Set)

- Stores elements in sorted (ascending) order.
- Does NOT allow null.
- Uses a Red-Black Tree internally.

```
import java.util.*;
```

```

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<Integer> ts = new TreeSet<>();

        ts.add(50);
        ts.add(10);
        ts.add(40);
        ts.add(20);

        System.out.println("TreeSet (Sorted): " + ts);
        System.out.println("First: " + ts.first());
        System.out.println("Last: " + ts.last());
    }
}

```

TreeSet (Sorted): [10, 20, 40, 50]

First: 10

Last: 50

11. Map Interface

Map is part of the Java Collection Framework, but NOT a subinterface of Collection. It stores data in key-value pairs.

Key Features

- Keys are unique (no duplicates)
- Values can be duplicated
- One key maps to one value
- Fast lookup using keys

Syntax: Map<KeyType, ValueType> map = new HashMap<>();

Common Implementations of Map

Implementation	Features
HashMap	Fast; no order maintained; allows one null key
LinkedHashMap	Maintains insertion order

TreeMap	Sorted by keys; no null key
Hashtable	Synchronized; no null key/value

Simple HashMap Example

```
import java.util.*;

public class MapExample {
    public static void main(String[] args) {
        Map<Integer, String> students = new HashMap<>();

        students.put(1, "John");
        students.put(2, "Alice");
        students.put(3, "Bob");
        students.put(2, "David"); // key 2 replaced with "David"

        System.out.println(students);
    }
}
```

Output (order not guaranteed): {1=John, 2=David, 3=Bob}

Example 2 — Accessing Values Using Keys

```
String name = students.get(1);
System.out.println(name); // Output: John
```

Example 3 — Looping Through a Map

Loop keys:

```
for (Integer key : students.keySet()) {
    System.out.println(key);
}
```

Loop values:

```
for (String value : students.values()) {
    System.out.println(value);
}
```

Loop key-value pairs:

```
for (Map.Entry<Integer, String> entry : students.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}
```

12. Queue Interface

Queue is a subinterface of Collection in Java. It represents a collection designed for holding elements before processing, typically in FIFO (First In, First Out) order.

Key Features of Queue

- FIFO order (first element added is the first removed)

- Duplicates allowed
- Allows null? — some implementations (like LinkedList) allow it; ArrayDeque and many others do NOT allow null

Common Implementations of Queue

Implementation	Features
LinkedList	Implements Queue; allows null
PriorityQueue	Sorted order; no null allowed
ArrayDeque	Fast; no null allowed; recommended replacement for Stack

Syntax: `Queue<Type> queue = new LinkedList<>();` or `Queue<Type> queue = new PriorityQueue<>();`

Example 1 — Basic Queue Operations (FIFO)

```
import java.util.*;

public class QueueExample {
    public static void main(String[] args) {
        Queue<String> q = new LinkedList<>();

        q.add("A");
        q.add("B");
        q.add("C");

        System.out.println(q);    // [A, B, C]
        System.out.println(q.poll()); // Removes A
        System.out.println(q);    // [B, C]
    }
}
```

Example 2 — Using PriorityQueue (Sorted Order)

```
Queue<Integer> pq = new PriorityQueue<>();

pq.add(30);
pq.add(10);
pq.add(20);

System.out.println(pq);    // Output order not guaranteed
System.out.println(pq.poll()); // Removes smallest element (10)
```

Note: a PriorityQueue's internal array/heap order isn't the same as its logical sorted order, so printing it directly doesn't necessarily show ascending order — but poll() always removes the smallest element first.

Example 3 — Useful Queue Methods

Method	Description
add()	Adds element (throws exception if fails)
offer()	Adds element (returns false if fails)

poll()	Removes head; returns null if empty
remove()	Removes head; throws exception if empty
peek()	Returns head without removing
element()	Returns head; throws exception if empty

Where Is Queue Used?

- Task scheduling
- Printer queue
- CPU job scheduling
- BFS (Breadth First Search in Graphs)
- Message handling

13. Cursors in Java

Java provides three types of cursors for iterating (looping) through collections:

1. Iterator
2. ListIterator
3. Enumeration

These are used to traverse elements of collections like List, Set, Vector, etc.

13.1 Iterator (One Way)

Iterator is a universal cursor used to traverse elements of any collection (List, Set, Queue).

- Works with all collection classes
- Moves forward only (one direction)
- Allows remove() during iteration
- Modern replacement for Enumeration

Methods: hasNext(), next(), remove()

[A] -> [B] -> [C] -> [D]

^

Cursor moves forward only ->

Iterator Example

```
import java.util.*;

public class IteratorExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        Iterator<String> it = list.iterator();
```

```

while (it.hasNext()) {
    String s = it.next();
    System.out.println(s);

    if (s.equals("Banana"))
        it.remove();
}

System.out.println("After remove: " + list);
}
}

```

Apple

Banana

Cherry

After remove: [Apple, Cherry]

13.2 ListIterator (Two Way)

ListIterator is a cursor used only for List (ArrayList, LinkedList) and supports two-way traversal.

- Works only with List
- Moves forward and backward
- Can add(), remove(), set() elements
- More powerful than Iterator

Methods: hasNext(), next(), hasPrevious(), previous(), add(), remove(), set()

```

<- Previous      Next ->
[A] <-> [B] <-> [C] <-> [D]
    ^

```

Cursor moves both ways

ListIterator Example

```

import java.util.*;

public class ListIteratorExample {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("One");
        list.add("Two");
        list.add("Three");

        ListIterator<String> lt = list.listIterator();

        System.out.println("Forward direction:");
        while (lt.hasNext()) {
            System.out.println(lt.next());
        }
    }
}

```

```

        System.out.println("Backward direction:");
        while (It.hasPrevious()) {
            System.out.println(It.previous());
        }
    }
}

```

Forward direction:

One

Two

Three

Backward direction:

Three

Two

One

13.3 Enumeration (Forward Only)

Enumeration is a legacy cursor used only for old classes like:

- Vector
- Stack
- Hashtable
- Works only with legacy classes
- Moves forward only
- Can't remove elements
- Oldest cursor (before Iterator)

Methods: `hasMoreElements()`, `nextElement()`

Enumeration Example

```

import java.util.*;

public class EnumerationExample {
    public static void main(String[] args) {
        Vector<String> v = new Vector<>();
        v.add("A");
        v.add("B");
        v.add("C");

        Enumeration<String> e = v.elements();

        while (e.hasMoreElements()) {
            System.out.println(e.nextElement());
        }
    }
}

```

A
B
C

Cursor Comparison — Quick Reference

Cursor	Works With	Direction	Can Modify?	Era
Iterator	Any collection (List, Set, Queue)	Forward only	remove() only	Modern (replaces Enumeration)
ListIterator	List only (ArrayList, LinkedList)	Forward and backward	add(), remove(), set()	Modern, most powerful
Enumeration	Legacy classes (Vector, Stack, Hashtable)	Forward only	Cannot modify	Oldest (legacy)

14. Tips for This Chapter

1. Keep the hierarchy in your head as a shape, not a list: Collection splits into List, Set, and Queue — but Map stands completely apart, NOT extending Collection at all. This single fact explains why Map can't be looped with a plain for-each the same way a List can (you loop its `keySet()`, `values()`, or `entrySet()` instead).
2. Collection (singular, interface) vs Collections (plural, utility class) is a genuine naming trap — say them out loud as 'a group of objects' vs 'a toolbox of static helper methods' to keep them separate in memory.
3. Choose List implementations by their trade-off: ArrayList for fast index-based reads, LinkedList for frequent insertions/deletions, Vector/Stack only when you specifically need legacy thread-safety (otherwise modern code prefers ArrayList + Collections.synchronizedList(), or Deque instead of Stack).
4. Choose Set implementations by what you need beyond 'no duplicates': HashSet for raw speed with no ordering guarantee, LinkedHashSet when you need uniqueness AND insertion order, TreeSet when you need uniqueness AND sorted order (but remember TreeSet rejects null).
5. For Map, always loop with the method matching what you actually need: `keySet()` when you only need keys, `values()` when you only need values, `entrySet()` when you need both together — `entrySet()` is more efficient than looping keys and calling `get()` separately for each one.
6. Queue is FIFO (first in, first out) — the opposite of Stack's LIFO (last in, first out). PriorityQueue breaks the FIFO rule on purpose: it always removes the smallest (or highest-priority) element next, regardless of insertion order.
7. The three cursors form a rough timeline: Enumeration (oldest, legacy-only, read-only) -> Iterator (modern, works everywhere, forward-only, can remove) -> ListIterator (most powerful, List-only, moves both directions, can add/remove/set). When in doubt for new code, Iterator is the safe general-purpose default.
8. Never call `list.remove()` directly on the collection while looping over it with a for-each or a plain Iterator without using the iterator's own `remove()` method — that throws a

ConcurrentModificationException. The Iterator example above shows the correct pattern: `it.remove()`, not `list.remove()`.

9. For HashSet, HashMap, and plain Queue/PriorityQueue printouts, never rely on the printed order matching insertion order or sorted order unless the specific implementation guarantees it (LinkedHashSet/LinkedHashMap for insertion order, TreeSet/TreeMap for sorted order). Printing a HashSet or HashMap directly is for debugging, not for producing a specific, reliable sequence.